

Documentation - Développement d'un outil de communication avec un analyseur de gaz

Valerio ANGIONE , Lenny SABINE , Vu Bao Chau DANG and
Elijah PEYRAT

Université Toulouse III Paul Sabatier

Licence Informatique

Encadrant : Gilles ATHIER (LAERO)

June 08, 2026

SOMMAIRE

1. Contexte et objectif	4
2. Installation des dépendances	4
3. Utilisation	4
3.1. Lancer l'application	4
3.2. Interface graphique	5
4. Structure du projet	5
5. Architecture générale	7
5.1. Séparation des threads	7
5.2. Rôle de la Queue	7
5.3. Flux de données	7
6. Description des modules	8
6.1. main.py	8
6.2. config.py	8
6.3. backend	9
6.3.1. serial_handler.py	9
6.3.2. data_processor.py	10
6.4. frontend	11
6.4.1. gui.py	11
6.4.2. plots.py	12
6.4.3. components.py	13

1. Contexte et objectif

Dans le cadre de l'UE Bureau d'Étude, l'ingénieur de recherche Gilles ATHIER du laboratoire LAERO (CNRS) nous a proposé de développer une application permettant aux techniciens de communiquer avec un analyseur d'ozone Thermo Fisher 49-i. L'objectif est de recevoir les données mesurées par l'appareil, de les afficher de manière claire et interactive sous forme de graphiques en temps réel, et de permettre la sauvegarde des mesures dans un fichier au format CSV.

L'application doit établir une communication série avec l'analyseur d'ozone, afficher les paramètres mesurés en temps réel (concentration d'ozone, températures, pression, débits, etc.), puis sauvegarder les données dans un fichier CSV.

2. Installation des dépendances

Dans un terminal —> `pip install -r requirements.txt`

Les bibliothèques suivantes sont requises :

Bibliothèque	Version	Utilisation
pyserial	≥ 3.5	Communication série
pandas	≥ 1.3	Stockage et manipulation des données
matplotlib	≥ 3.4	Affichage des graphiques
tkinter	—	Interface graphique (inclus dans Python)

3. Utilisation

3.1. Lancer l'application

Dans un terminal —> `python3 main.py`

3.2. Interface graphique

Une fois l'application lancée :

- Cliquer sur ► **Start acquisition** pour démarrer la communication série et l'affichage des données en temps réel.
- Cliquer sur  **Stop acquisition** pour arrêter l'acquisition sans fermer l'application.
- Cliquer sur  **Save as CSV** pour sauvegarder manuellement les données dans un fichier CSV à l'emplacement souhaité.
- Cliquer sur  **Refresh** pour redessiner manuellement tous les graphiques.

4. Structure du projet

La structure des fichiers est détaillée dans la figure 1 ci-dessous. Le projet est organisé en trois parties distinctes :

- **Point d'entrée** : `main.py` initialise les composants principaux (Queue, backend, interface graphique) et démarre la boucle d'événements Tk. `config.py` centralise tous les paramètres de configuration (port série, baudrate, intervalle d'acquisition).
- **Backend** (`backend/`) : responsable de la communication série avec l'analyseur. `serial_handler.py` gère la connexion et l'acquisition des données sur un thread en arrière-plan. `data_processor.py` parse les trames brutes reçues en dictionnaires typés.
- **Frontend** (`frontend/`) : responsable de l'affichage des données. `gui.py` gère l'interface graphique principale et le polling de la file d'attente. `plots.py`

contient les fonctions de rendu des graphiques matplotlib. `components.py` fournit des composants Tkinter réutilisables.

- **Sauvegarde** (`record/`) : dossier généré automatiquement à la racine du projet, contenant les fichiers CSV horodatés produits lors de chaque session d'acquisition.

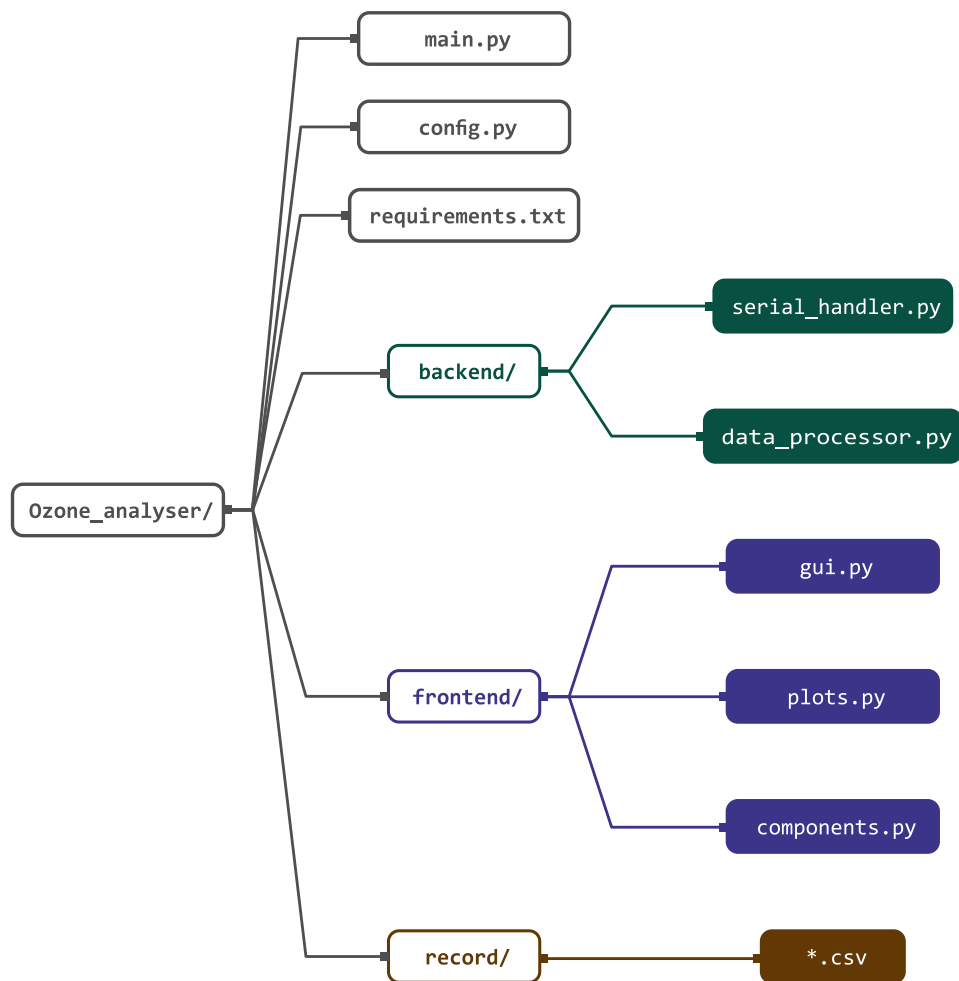


Figure 1: La structure du projet

5. Architecture générale

L'application repose sur une architecture producteur-consommateur organisée autour de trois composants principaux : le backend (`SerialHandler`), la file d'attente (`Queue`) et l'interface graphique (`GraphApp`).

5.1. Séparation des threads

L'application utilise deux threads distincts afin de garantir la réactivité de l'interface graphique :

- **Thread en arrière-plan** (`SerialHandler`) : communique avec l'analyseur via le port série, envoie les commandes et reçoit les trames de données.
- **Thread principal** (`Tk`) : gère l'interface graphique, le polling de la file d'attente et la sauvegarde CSV. Toutes les opérations sur le `DataFrame` et les graphiques sont réalisées sur ce thread.

5.2. Rôle de la Queue

La Queue est le seul canal de communication entre les deux threads. Elle est nativement thread-safe, ce qui évite tout risque de *race condition* sans nécessiter de verrou manuel. Sa taille est limitée à 200 entrées. Si elle est pleine, la donnée la plus ancienne est supprimée via une politique drop-oldest, garantissant que l'application affiche toujours les mesures les plus récentes.

5.3. Flux de données

Le flux de données suit le chemin suivant :

1. Le `SerialHandler` envoie `lrec 1 1` à l'analyseur et reçoit une trame brute.
2. La trame est validée via `_is_data_frame()` et insérée dans la Queue via `put_nowait()`.

3. Toutes les 200ms, GraphApp vide la Queue via `get_nowait()` dans `_poll_queue()`.
4. Chaque trame est parsée via `process_raw_data()` et ajoutée au DataFrame.
5. Le DataFrame est sauvegardé automatiquement dans `record/*.csv` via `_autosave()`.
6. Les graphiques sont redessinés via `refresh_all()`.

6. Description des modules

6.1. main.py

- `main.py` : point d'entrée de l'application, initialise les composants et démarre la boucle d'événements Tk.
- Queue : créée par `main()`, sert de canal de communication thread-safe entre le backend et l'interface graphique.
- GraphApp : interface graphique principale, reçoit les données de la Queue et les affiche sous forme de graphiques.
- `on_close()` : gestionnaire de fermeture, arrête proprement le backend via `backend.stop()` avant de détruire la fenêtre.

6.2. config.py

- `config.py` : fichier de configuration, contient les paramètres de l'application.
- AppConfig : classe de configuration, définit les paramètres de connexion série (PORT, BAUDRATE, ID_ANALYSEUR), d'acquisition (ACQUISITION_INTERVAL, MAX_DATA_POINTS) et de la file d'attente (QUEUE_MAXSIZE).
- PLOT_CONFIGS : dictionnaire contenant la configuration des graphiques (couleur, titre, colonnes du DataFrame à afficher).

6.3. backend

6.3.1. serial_handler.py

- **class** SerialHandler(data_queue, debug=True)

Gère la communication série avec l'analyseur d'ozone. Lance un thread en arrière-plan pour l'acquisition des données et pousse les trames brutes dans la file d'attente.

- **méthode** SerialHandler.connect(port, baudrate, device_id) → bool

Ouvre le port série et active le mode remote via _set_remote_mode(). Retourne True si la connexion est établie, False sinon.

- **méthode** SerialHandler._set_remote_mode(device_id) → bool

Envoie la commande set mode remote et attend la confirmation set mode remote ok. Retourne True si confirmé dans les 12 tentatives, False sinon.

- **méthode** SerialHandler._send_command(cmd, device_id) → None

Encode et envoie une commande sur le port série au format <id><cmd>\r\n.

- **méthode** SerialHandler.read_response() → str

Lit une ligne sur le port série. Bloque jusqu'à réception d'une réponse non vide ou expiration du délai (8 secondes). Retourne la ligne décodée.

- **méthode** SerialHandler.start_acquisition(port, baudrate, id_analyseur, interval) → bool

Initialise la connexion série et démarre le thread `_acquisition_loop()`.
Retourne True si le démarrage est réussi.

- **méthode** `SerialHandler._acquisition_loop(device_id, interval) → None`

Boucle principale du thread en arrière-plan. Envoie `lrec` à intervalles réguliers, valide la trame reçue via `_is_data_frame()` et l'insère dans la file d'attente via `_enqueue()`.

- **méthode** `SerialHandler._is_data_frame(line) → bool`

Vérifie qu'une ligne contient tous les champs attendus (o3, flags, pres, etc.). Retourne True si la trame est valide.

- **méthode** `SerialHandler._enqueue(raw_data) → None`

Insère une trame dans la file d'attente via `put_nowait()`. Si la file est pleine, supprime la donnée la plus ancienne avant d'insérer la nouvelle.

- **méthode** `SerialHandler.stop() → None`

Arrête le thread d'acquisition via `stop_event` et ferme le port série.

6.3.2. *data_processor.py*

- **module** `data_processor.py`

Parse les trames brutes reçues depuis la file d'attente en dictionnaires typés, prêts à être ajoutés au DataFrame.

- **fonction** `process_raw_data(raw) → dict | None`

Reçoit une trame brute sous forme de chaîne de caractères, extrait chaque champ via `re.search()` et retourne un dictionnaire contenant les valeurs parsées (`o3`, `cellA`, `cellB`, `benchT`, `lampT`, `o3lamp`, `flowA`, `flowB`, `pression`). Retourne `None` si la trame est invalide.

- **fonction** `get(label) → float | None`

Fonction auxiliaire interne à `process_raw_data()`. Recherche la valeur associée à un label donné dans la trame brute via une expression régulière. Retourne `None` si le label est absent ou si la valeur ne peut pas être convertie en `float`.

6.4. *frontend*

6.4.1. *gui.py*

- **class** `GraphApp(root, data_queue, backend, config, interval, max_points=500)`

Interface graphique principale de l'application. Gère l'affichage des graphiques, le polling de la file d'attente, la sauvegarde CSV et les boutons de contrôle.

- **méthode** `GraphApp.create_interface() → None`

Crée les onglets de graphiques, la barre de boutons (Start, Stop, Save, Refresh) et le label de statut.

- **méthode** `GraphApp._start_acquisition() → None`

Appelée par le bouton Start. Lance l'acquisition via `backend.start_acquisition()` et met à jour l'état des boutons.

- **méthode** `GraphApp._stop_acquisition()` → None

Appelée par le bouton Stop. Arrête l'acquisition via `backend.stop()` et met à jour l'état des boutons.

- **méthode** `GraphApp.schedule_poll()` → None

Planifie l'appel de `_poll_queue()` dans 200ms via `root.after()`.

- **méthode** `GraphApp._poll_queue()` → None

Vide la file d'attente via `get_nowait()`, parse chaque trame via `process_raw_data()`, ajoute les nouvelles lignes au `DataFrame`, déclenche la sauvegarde automatique et rafraîchit les graphiques. Se replanifie automatiquement via `schedule_poll()`.

- **méthode** `GraphApp._autosave()` → None

Sauvegarde automatiquement le `DataFrame` dans un fichier CSV horodaté dans le dossier `record/` à chaque nouvelle donnée reçue.

- **méthode** `GraphApp.save_csv()` → None

Appelée par le bouton Save. Ouvre une boîte de dialogue pour choisir l'emplacement du fichier et sauvegarde le `DataFrame` au format CSV.

- **méthode** `GraphApp.refresh_all()` → None

Redessine tous les graphiques de tous les onglets à partir des données actuelles du `DataFrame`.

6.4.2. *plots.py*

- **module** `plots.py`

Contient les fonctions de rendu des graphiques matplotlib utilisées par GraphApp.

- **fonction** `_x_axis(df) → Series`

Retourne la colonne timestamp du DataFrame si elle existe, sinon retourne l'index du DataFrame.

- **fonction** `_format_time_axis(ax, df) → None`

Formate l'axe des abscisses au format HH:MM:SS via `mdates.DateFormatter`.

- **fonction** `create_single_plot(ax, df, config) → None`

Dessine un graphique à une seule courbe à partir de la colonne définie dans `config`. Affiche un message d'attente si le DataFrame est vide.

- **fonction** `create_dual_plot(ax, df, config) → None`

Dessine un graphique à deux courbes à partir des colonnes définies dans `config`. Affiche un message d'attente si le DataFrame est vide.

6.4.3. *components.py*

- **module** `components.py`

Contient les composants Tkinter réutilisables de l'interface graphique.

- **class** `Tooltip(widget, text)`

Affiche une infobulle au survol d'un widget Tkinter. La fenêtre apparaît sous le widget et disparaît lorsque la souris le quitte.

- **méthode** `Tooltip._show(event=None) → None`

Crée et affiche la fenêtre de l'infobulle au survol du widget.

- **méthode** `Tooltip._hide(event=None) → None`

Détruit la fenêtre de l'infobulle lorsque la souris quitte le widget.